rogii-wellbore-geology-prediction

8th place - PF ranker + Synthetic-Pretrained U-Net

Rank	8
Team	富士山
Author	Andrey Chankin
Topic ID	733281
Version	Original

Source: https://www.kaggle.com/competitions/rogii-wellbore-geology-prediction/discussion/733281
Retrieved with Kaggle CLI on 2026-08-07.

Andrey's part

Hi guys!

During the competition I ve read a tones of ai slop (mostly produced by myself), so I will write this manually, not as a final solution writeup, but as how was the progress done. If something will be unclear - feel free to ask below.

TL:DR

My approach was 20 various PF with supervised ranker using them and some positional, geological features.

Problem statement

I have joined in the 1st part of June and until the middle of July had no solution better then constant prediction. Mainly because I was not understanding how to approach the problem. So time was spend reading forum, watching demo of the starsteer, researching data,

Manual matching soft

For starsteer I decided I want to do something similar and vibecoded manual well matching software. You could upload a well with a rank based on how well constant predicts it (1 = worst prediction), look at it gr, tvt, typewell gr, selfreverse gr, metrics and try to choose a window size, scale, smooth, offset yourself to match the signals.



Looks like a nice and easy match, ye (the right small image is the horizontall well + prediction)? But in reality it was easy to match only "strong" movements TVT, when "slight" (which were the most) were barely recognizable.

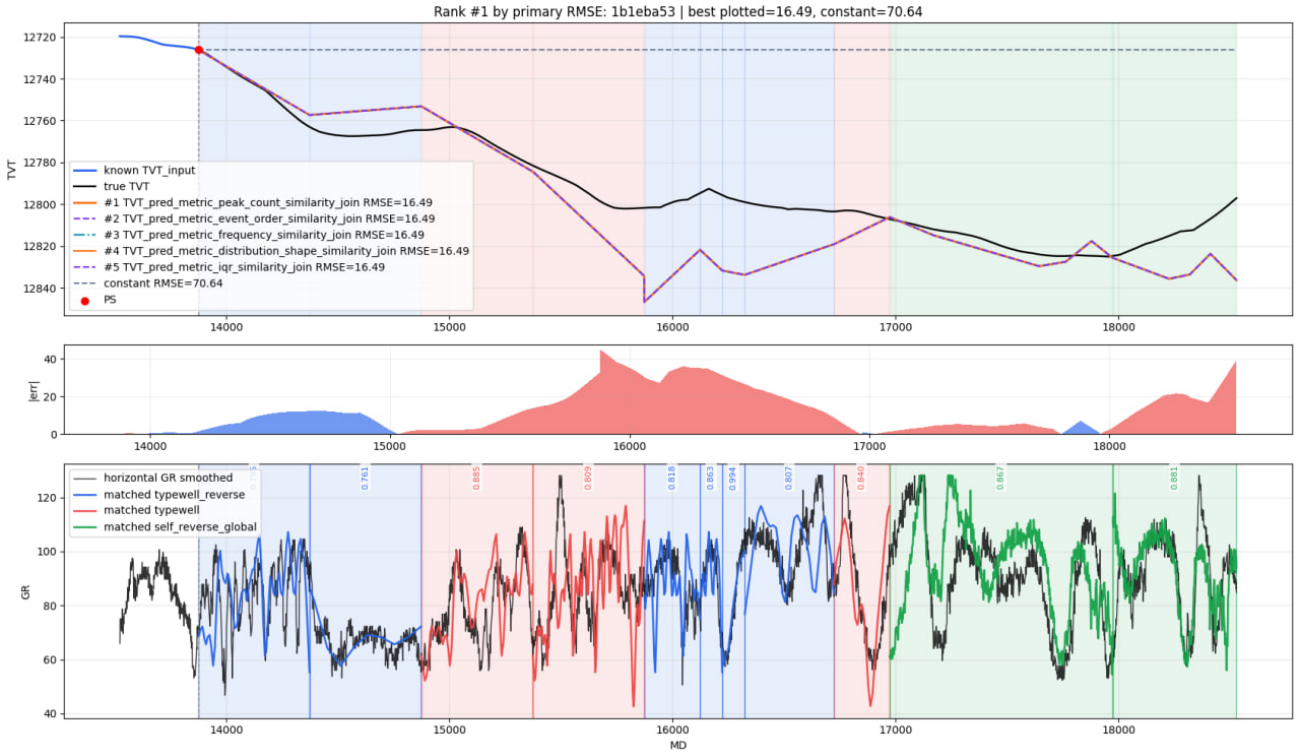
Below the example of "slight" movement matching, on the TVT it may look nice, but blindly matching it by GR's was barely possible without knowing the ground truth.

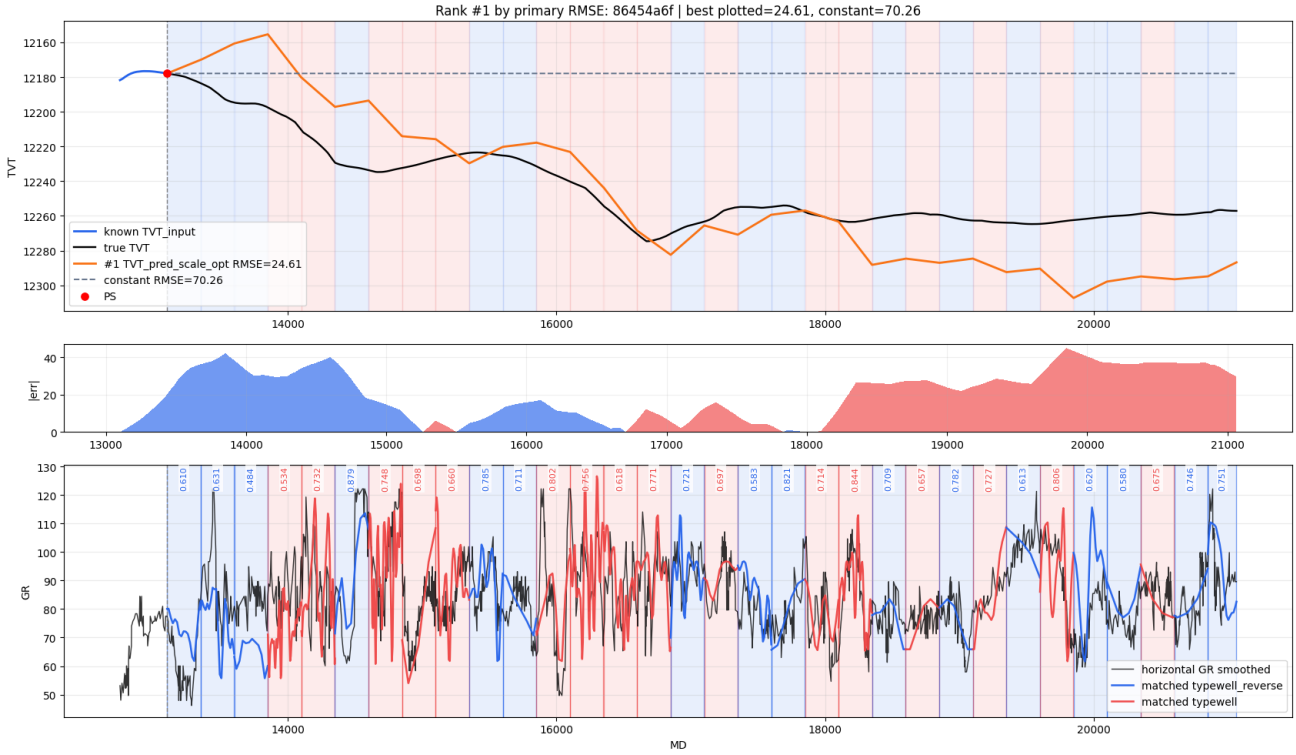


First models

So, based on the approach above I have tried to make an automated pattern matcher. I spend a while designing a metric, testing different ones, but at the end of a day it was maybe slightly better then constant prediction. For some "strong" movement wells it was ok sometimes, but for regular wells it was pretty bad.

Here are some examples of the worst constant-prediction wells:





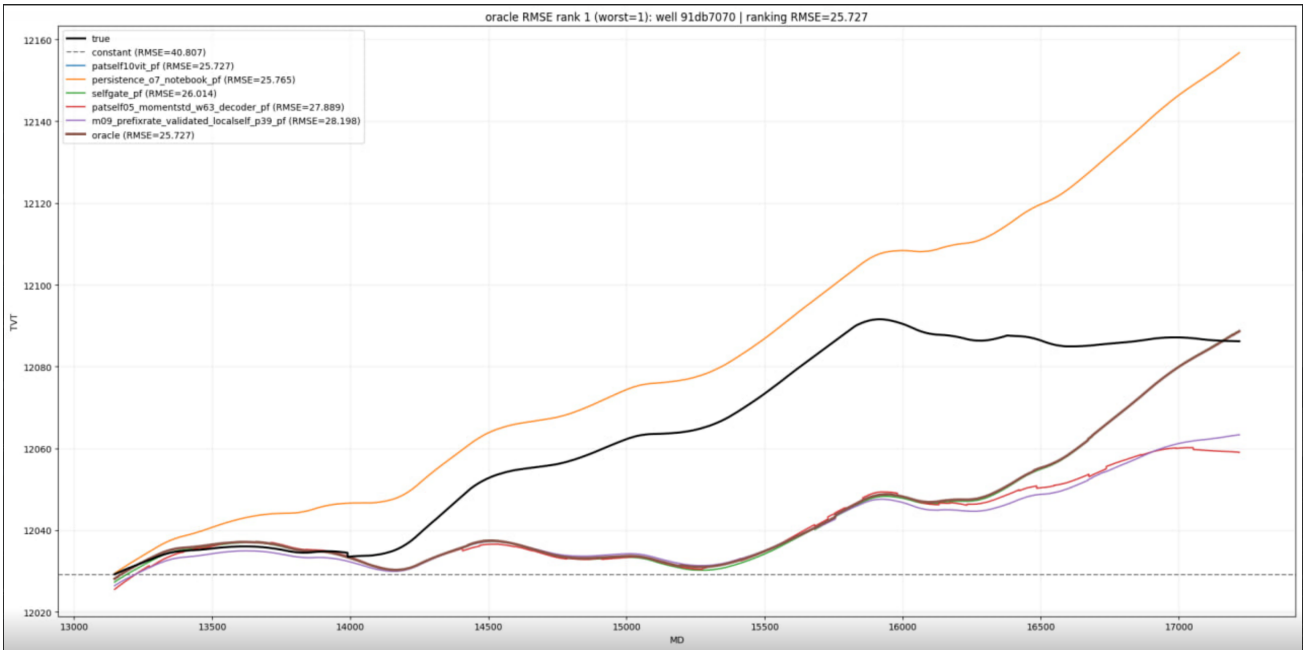
Breakthrough

PF

About the middle of July I decided that my approach is the dead end. Also a lot of problem understanding came from @shreygandhi early writeup, tbh it was the only one I read because he was top 6 at that time.

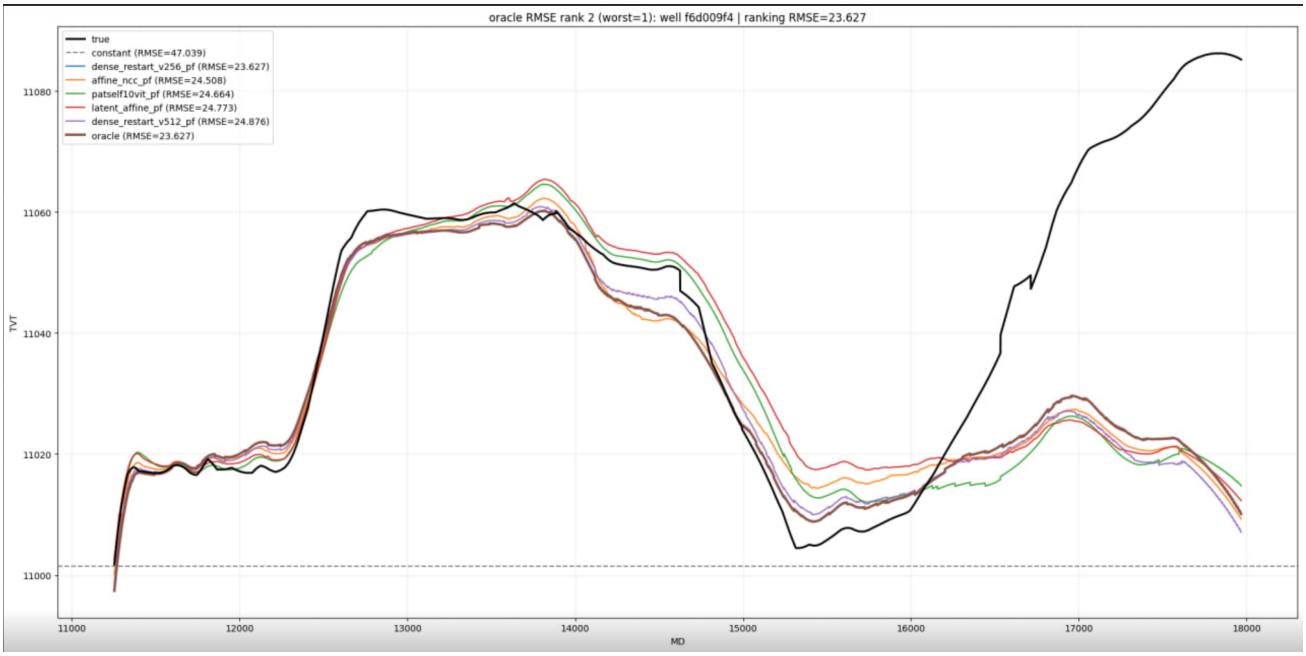
So I decided to switch to PF, here actual understanding of the pattern matching helped in many cases. My approach was designing different types of PF's. The bank included GR/type-well matching, self-reference, spatial-neighbour, filtering, smoothing, change-point and probabilistic tracking approaches.

Idea of making decorrelated error or different signal errors came from looking at their highest errors wells.



Here the orange PF could be a good blend with other PF's, but on "slight" movement wells it was very bad, scoring 55 RMSE.

But for some wells there were still no good candidate for blend:



How to choose PF's for blend

At the moment I had already about 30 PF's, running full candidates estimators would not be possible anymore. From @shreygandhi writeup I took the idea of estimating oracle-type metric and adding PF's in a greedy way.

Oracle-types

Firstly I ve been estimating oracle with whole well single PF selector, but then remembered my experience with manual matching and decided to switch to 256 MD window, because some various PF's can predict various pieces of well.

Obviously window oracle was better.

My first experiments with 7 PF bank had metrics like:

Whole-well Oracle

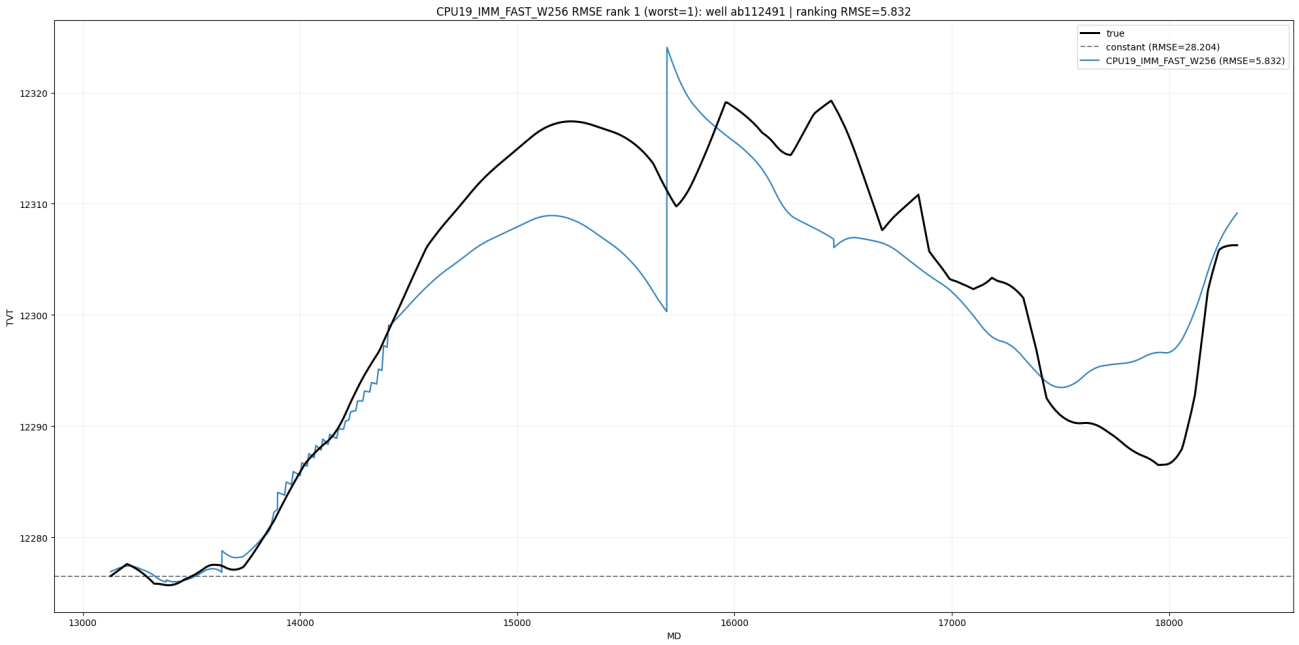
Bank name	N	Overall	Mean	Median	P90	P95	Max
O7	7	3.982578	2.987096	2.326777	5.535167	6.837023	26.014447
CPU20	20	**3.055603**	**2.505059**	**2.074897**	**4.673471**	**5.539533**	**12.162778**

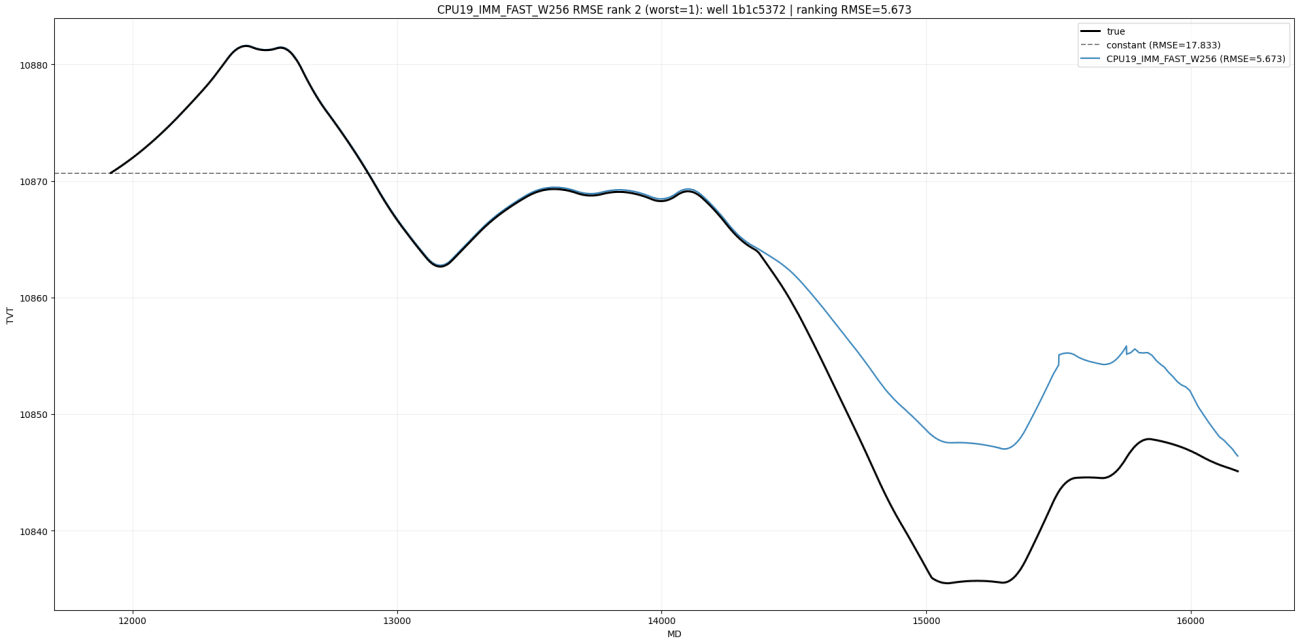
W256 Oracle

Bank name	N	Overall	Mean	Median	P90	P95	Max
O7	7	2.886406	1.976838	1.386853	3.959398	5.170809	25.858313
CPU20	20	**1.276588**	**1.058108**	**0.885115**	**1.877131**	**2.406464**	**5.832096**

You may consider PF bank names odd: O7 - means best overall 7 at the time of research CPU20 - its 20 PF's bank rewritten for CPU inference (to have full reproducibility)

Below some plots of different PF banks oracle for worst prediction wells, they look odd and unrealistic to be a real well:





As you can see some wells were unpredictable with my set of PF's or some unrealistic trajectories has been selected.

Ranker

I decided that simple blend should be replaced with ranker supervised model to choose which PF to use for each segment.

Features beyond the PF bank

The 20 PF trajectories were supplemented with several groups of inference-time features. Different selector models used different subsets:

- **Trajectory dynamics:** slope, displacement and local variation.
- **PF disagreement:** spread and distance between candidates.
- **GR matching:** agreement with type-well and observed GR.
- **Well geometry:** depth, vertical movement and relative position.
- **Surface context:** agreement with nearby geological surfaces.
- **Temporal context:** changes between consecutive well segments.
- **Spatial reliability:** historical PF performance in nearby wells.
- **Whole-well context:** global statistics and early-to-late drift.

Performance-based priors were computed from training-fold wells only.

Models tested

This part I had to put on autoresearch for LLM, because it was not realistic to test everything manually. In total it was +- 40 different models. Some of them below:

- **TCNs:** lag, peak and dual-head variants.
- **GRU / PF-MoE:** temporal and pairwise selectors.

- **CatBoost:** ranking and nested-risk models.
- **Type-well networks:** MoE, Perceiver and Transformer.
- **Tree ensembles:** LightGBM and Extra Trees.
- **Direct models:** axial Transformer and WellDirect.
- **Other baselines:** HMM, TabM, FT-Transformer and RBF-SVR.

Closer to the end I had to cut this list significantly keeping only CPU-inference models.

So most of the best experiments have been tested only with:

- **Hierarchical LightGBM ranker**
- **Surface-aware LightGBM:** predicts a smooth trajectory using geological surface context
- **CatBoost ranker**
- **Hierarchical GRU:** learns dependencies between consecutive well segments.
- **Temporal CNN:** detects local and long-range GR–typewell patterns.

approach with LGBM+GRU blend of 7 PF's gave me the first position in gold with a ~6.0.

Merge

At this point I was unconfident with my solution sticking at the gold until the end of the competition and I decided to merge with looking for merge teams with quick inference solutions, because mine was taking 6.5 hours to score. I merged with @sggpls and he added @yuto0712

My bug

It turned out to be the best idea because Sergey found an odd bug in my best solutions: I trained a model with one set of PF's, but at the inference one of them was replaced with a different one and it made a a jump on LB to the gold zone xD.

I have started to investigate the problem and it happened that the wrong PF was actually that one orange from the above picture with 55 RMSE.

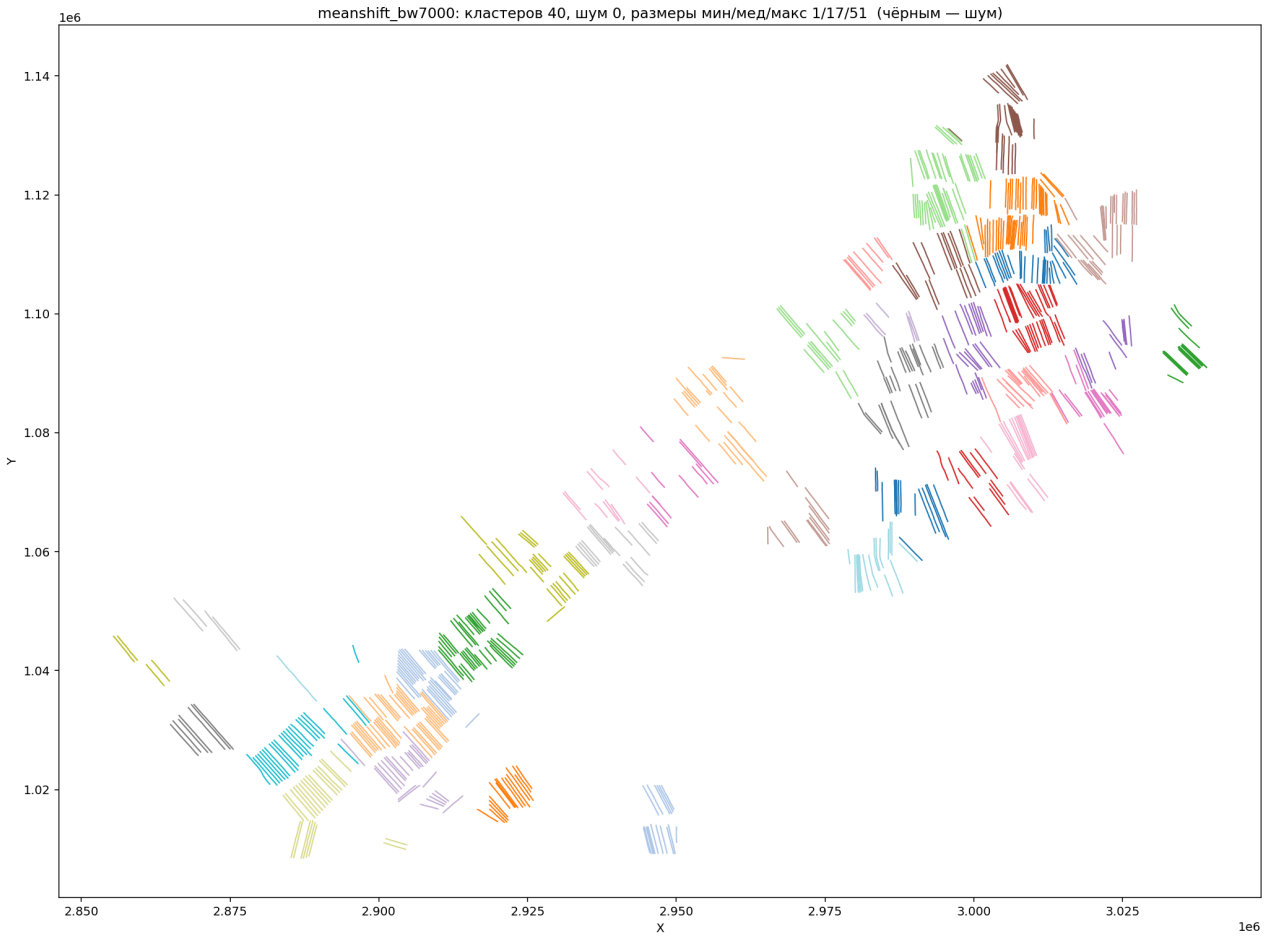
Probably thats because in the public test "high" movement well has been present and regular PF's could not predict it well.

After this I became paranoid and were testing server/kaggle reproducibility for 3 times, yet still once submission failed because LLM did put a hardcode check for 3 test wells.

CV

My initial CV was group K fold, I have splitted wells based on their geographical position and put the closest wells into a single group, later the group was fully used either in train or in test.

Sergey said that group stratified CV had better correlation with LB for his models.



Tuning

Not much of it has been done actually due to the time limitations.

Final ensemble

I trained five selectors on the CPU20. To avoid overfitting the ensemble, weights were fitted on four whole-well folds and evaluated on the held-out fifth fold.

The best Overall blend contained three complementary models:

- **Hierarchical LightGBM:** 0.399
- **Surface-trend LightGBM:** 0.129
- **Temporal TCN:** 0.472

OOF metrics for stratified CV were:

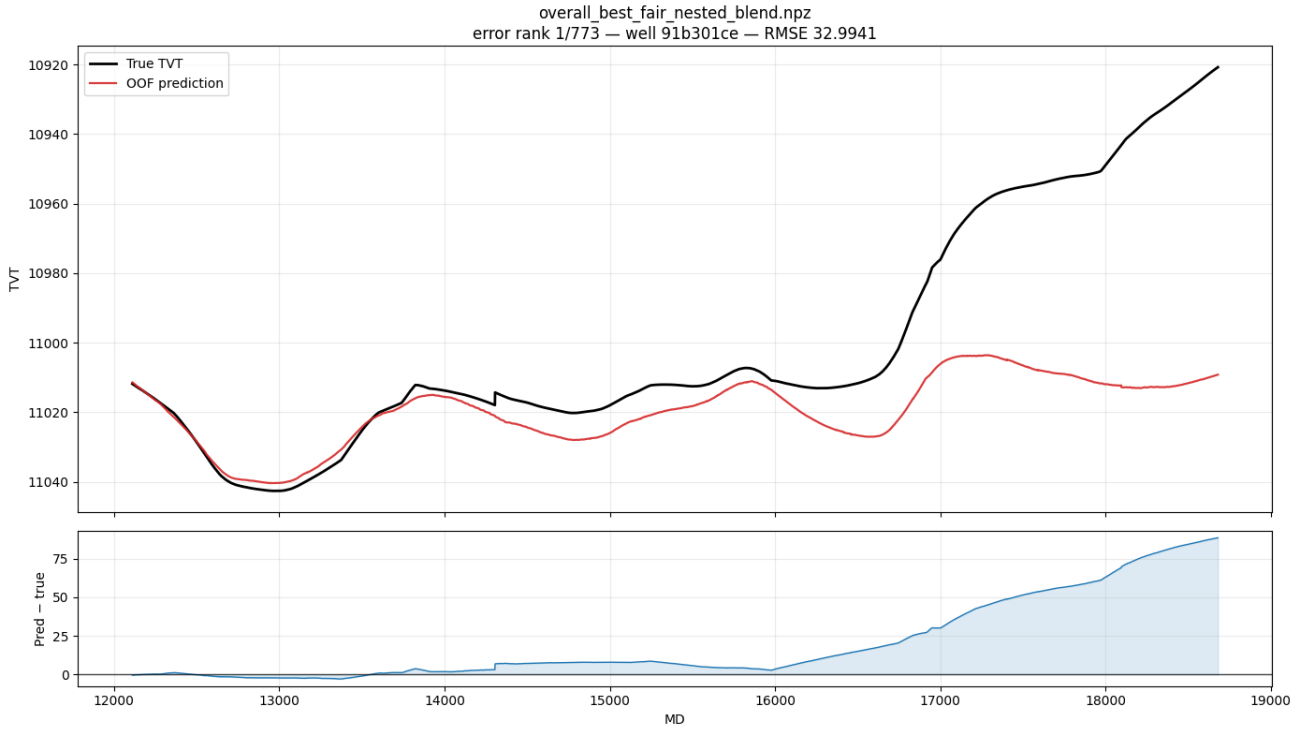
Metric	Value
Overall RMSE	5.70874
Median well RMSE	2.91723
P90	8.50775
P95	11.17887

CatBoost and GRU remained useful standalone models, but adding them did not improve the honest Overall blend.

How the selector worked

Each model scored the 20 PF candidates over well segments. The scores were converted into blending weights and interpolated along the well. It was done to have a smooth prediction and avoid heavy errors done due to the incorrect selection.

Below some worst predicted wells:



As you can see the final predictions dont have dramatic TVT changes or unrealistic trajectories.

LB results

Our last submissions scored 1 hour before the deadline and they were pretty bad on public LB, so we had a long argument about which submit to choose, at the last minute decided to pick one purely on CV and it paid off, because all 5.5 public went to 6.7-7.0 on private.

Appreciation's

I would like to thank our teamleader @sggpls for doing tremendous amount of work in the last week and managing our team.

@yuto0712 for unique solution which helped in the final blend

@shreygandhi for his writeup

and @alin256 who was not a participant, but actually a scientist whose papers I have researched to understand the problem, and take ideas from them. To be precise:

- Direct Multi-Modal Inversion of Geophysical Logs Using Deep Learning
- Geosteering Robot Powered by Multiple Probabilistic Interpretation and Artificial Intelligence
- NORCE's open-source digital geosteering projects
- High-Precision Geosteering via Reinforcement Learning and Particle Filters

Yuto's part — Synthetic-pretrained U-Net

This is the U-Net leg of our 8th place solution. The final submission was `0.5 × b230_cpu20 (@bluepill) + 0.5 × residual_spatial5`, and the latter is my 5-fold U-Net bank, which @sggpls extended with his tail-25 residual correction and ms7000 spatial folds. **What follows covers the U-Net itself only** — the residual correction, the fold definitions, the blend, and the PF side are all other members' work.

TL;DR

- **The error is per-well dip lock-in, not tail difficulty.** The last 25% of a well accounts for 41.6% of total SSE, but only because dip error accumulates. Oracle-removing a per-well linear trend takes 6.44 → 3.33.
- **A synthetic generator that invents the geological surface `u = TVT + Z`** gave zero-shot 7.62 after 200 epochs that never touch a real well. Inventing TVT directly is caught by a domain gate at AUC 1.0.
- **Dropping pretrain → finetune in favour of keeping 50% synthetic flowing throughout real-data training** broke the 6.6 wall and reached 5.06.
- **The pretraining seed is a lottery, and the pretraining score cannot screen it.** Re-rolling took a single fold from 9.081 → 6.496.
- **The same recipe fails on different wells when you change the fold split** (residual correlation 0.826). Mixing in 30% of a group scheme that is 0.41 worse on average beats stratified alone.

U-Net setup

Item	Value
Representation	One image per well. `H=384` TVT lattice rows × `W=768` MD columns, `dh=0.5 ft` (±96 ft), `dw=16 ft`
Backbone	ResNet34 U-Net (smp), 9 input channels, InstanceNorm(5ch)
Loss	Column-wise cross-entropy
Inference	Decode the posterior expectation over the lattice, `temp=1.0`

Item	Value
Training	Two stages: 200ep synthetic-only → 50ep real 50% + synthetic 50%

Error structure

An early error breakdown set the direction for everything that followed.

Position within well	RMSE	Share of total SSE
0.0–0.2	2.95	4.2%
0.2–0.4	4.77	11.0%
0.4–0.6	6.10	18.0%
0.6–0.8	7.23	25.2%
0.8–1.0	**9.28**	**41.6%**

Error grows almost monotonically from 2.95 at the entry to 9.28 at the toe. An oracle decomposition identifies the cause.

Oracle operation	pooled RMSE
Baseline	6.44
Remove per-well constant offset	4.58
Remove per-well linear trend	**3.33**
Predict the last 25% perfectly	4.61

The tail is not intrinsically hard — a dip error spanning the whole well accumulates and surfaces at the tail. Fixing only the tail is capped at 4.61, which is worse than removing a constant across the whole well (4.58).

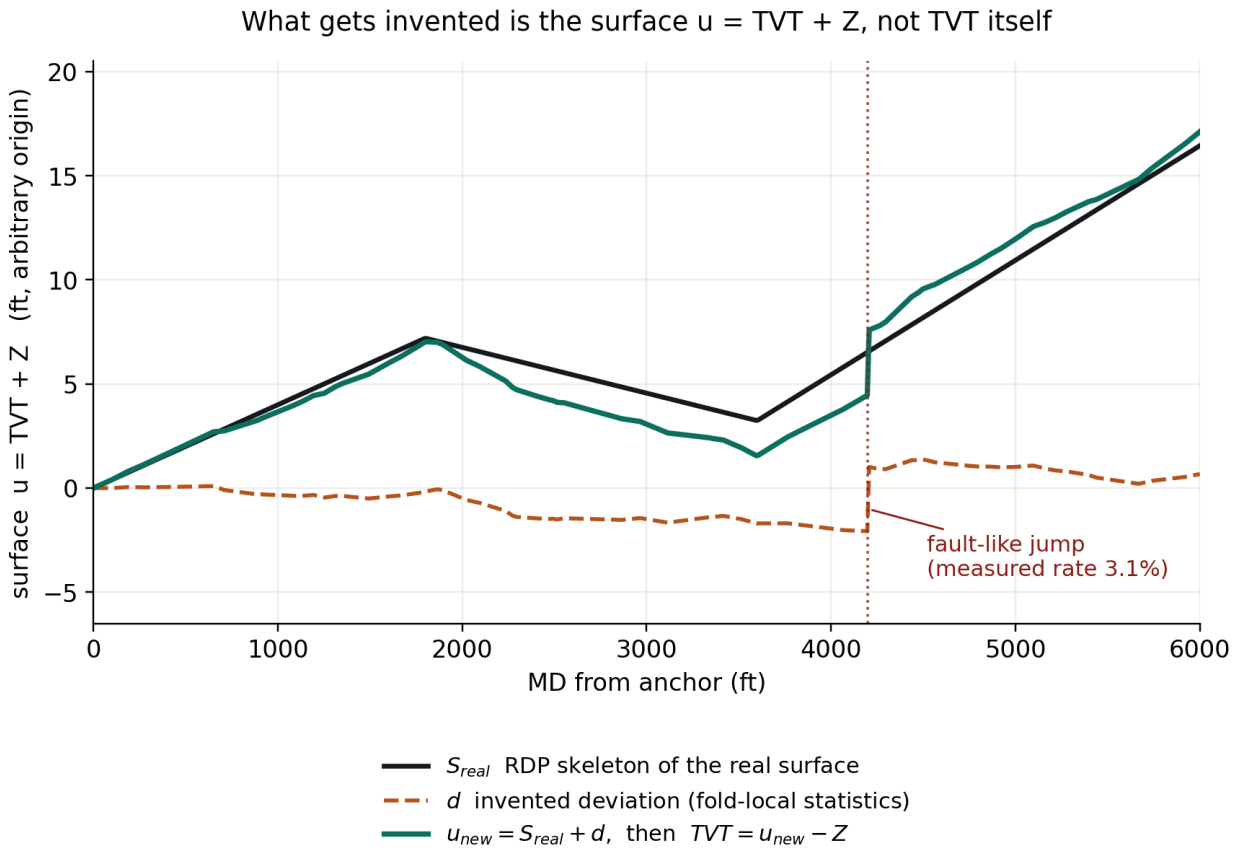
Synthetic data: inventing the geological surface

The training data has 773 wells, so each fold of a 5-fold split trains on roughly 618 of them. An image CNN does not learn "matching" — it memorises a **vocabulary of geological surfaces**. My initial generator only rescaled real surfaces, so each fold's model saw the same ~618 surfaces every epoch.

Inventing TVT directly failed. I implemented a domain gate that discriminates real from synthetic, and it separated them at **AUC 1.0 using surface curvature alone**.

The cause is physical. Drillers steer Z to follow the formation, which couples `dTVT ≈ -dZ`. Inventing TVT alone breaks that coupling and roughens the surface. But inventing the surface `u = TVT + Z` completely independently is no better: with the real trajectory fixed, the wellbore drifts ~100 ft off the typewell and stops being coherent geosteering.

The construction that worked is `u_new = S_real + d`.



Component	Content
`S_real`	RDP skeleton of a real surface (piecewise linear, consistent with the trajectory, real control points preserved)
`d`	Piecewise-linear deviation field built only from the fold's training-side wells
`TVT_new`	`u_new - Z`. Rejected if outside the real TVT drift envelope, resampled up to 12 times

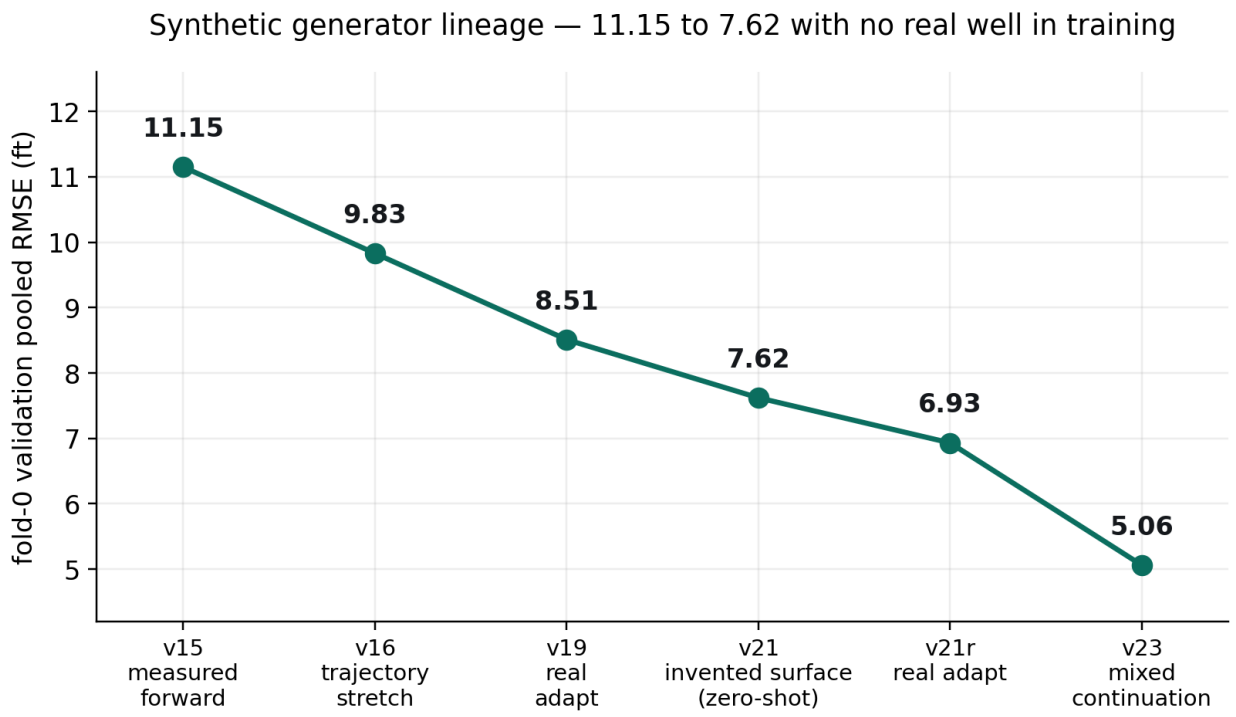
The fold-local statistics behind `d`:

Statistic	Value
Segment length	Empirical distribution (median ≈ 98 ft)
Slope deviation	AR(1), rho ≈ 0.248, empirical innovations
PS transition slope	Measured distribution
Fault-like jumps	Measured rate 3.1%

On top of that, `decoy_prob = 0.25` injects a **false match at ±15 ft**. That is the Eagle Ford aliasing period, so this is adversarial training against the dominant failure mode — locking onto an adjacent bed.

The real trajectory and prefix annotations stay real; only the horizontal GR is re-rendered along the new path, preserving per-row observation residuals and missingness patterns.

Training with `synth_prob=1` (never showing a real well) for 200 epochs gives **zero-shot 7.62**.



Training: replacing pretrain → finetune with mixed continuation

The standard recipe — pretrain on synthetic, finetune on real — **plateaued at 6.6**. Instead, I kept 50% synthetic flowing throughout real-data training.

```
--synth-prob 0.5   --skeleton-eps 2.0   --decoy-prob 0.25
--hflip-prob 0.5   --anchor-real-prob 0.20
--lr 3e-5   --epochs 50   --train-scope all   --freeze-batchnorm-stats
```

Single-variable comparison from the same fold and the same pretrain:

fold	synth_prob=0	synth_prob=0.5
f0	7.412	**6.919**
f1	6.758	**6.496**
f2	5.390	**5.226**

Fold 0 eventually reached 5.06. My reading is that the synthetic stream keeps working as a regulariser against overfitting to the 618 wells all the way to the end of training, rather than being something you discard at handover.

Variance from the pretraining seed

One fold sat 2 ft worse than the others (9.081). **Re-rolling only the pretraining seed brought it to 6.496 (−2.59)** — same recipe, same statistics, same validation wells.

The problem is that you cannot identify a good seed from its pretraining score.

Pretrain best-val	After mixed continuation
9.959	**6.76**
9.417	**6.50**
9.722	9.08
10.243	8.73

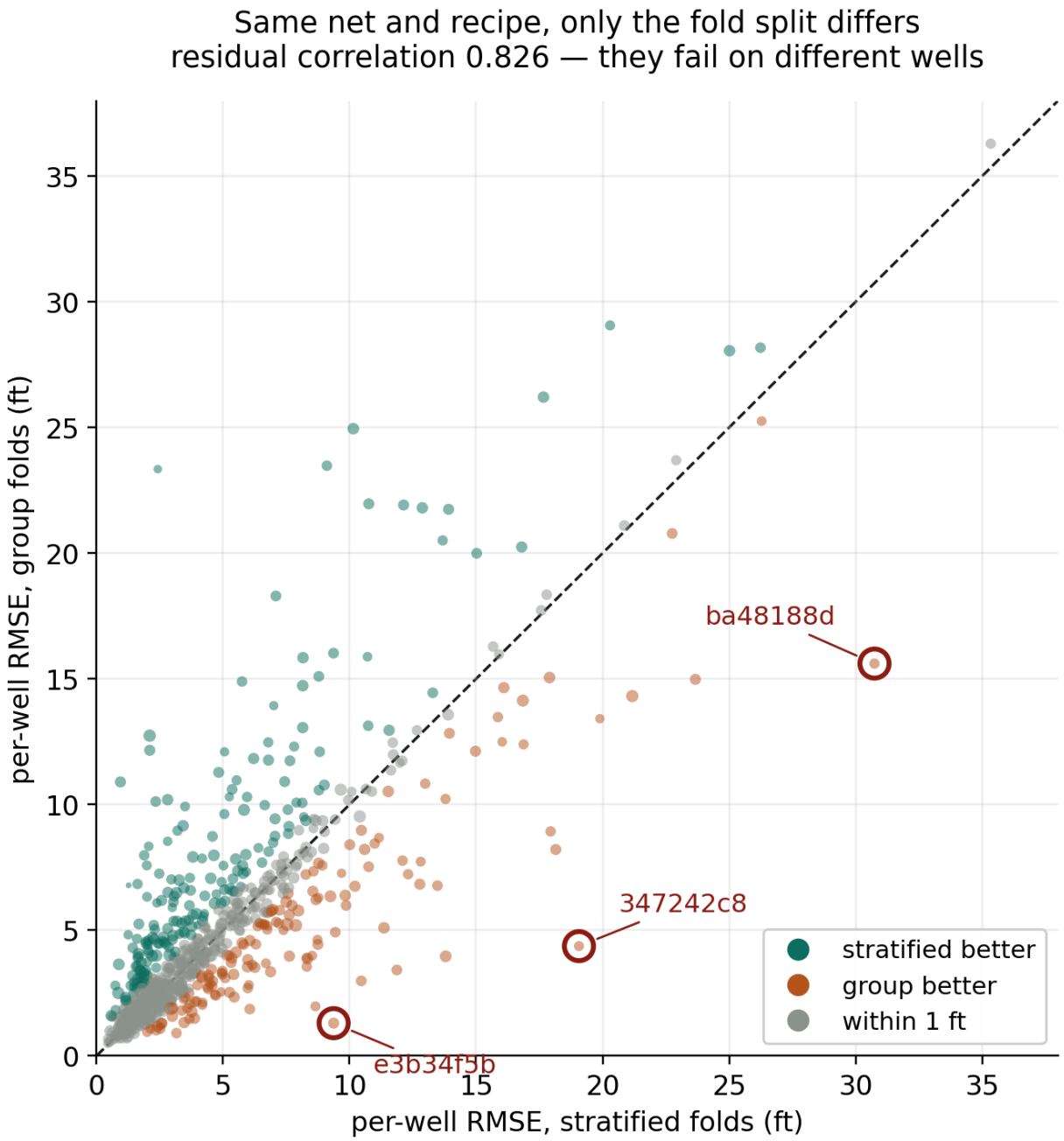
The two are uncorrelated. The noise floor is stage-dependent: ImageNet-initialised pretraining spans 0.955 ft at ep10 for the same seed, while mixed continuation spans 0.008 ft. Almost all the variance comes from the pretraining draw.

Two practical takeaways. **Do not screen seeds by pretraining score. If a fold is 2+ ft worse than the others, suspect an outlier draw rather than a hard fold.**

Validation schemes and diversity

Using the validation schemes @sggpls built on 40 spatial clusters (MeanShift bw=7000), I trained **both a stratified 5-fold and a group 5-fold with an identical recipe**. Both give full 773-well, non-overlapping out-of-fold coverage.

	pooled CV	well median	>12ft
stratified 5-fold	6.437	3.486	46/773
group 5-fold	6.847	3.828	56/773

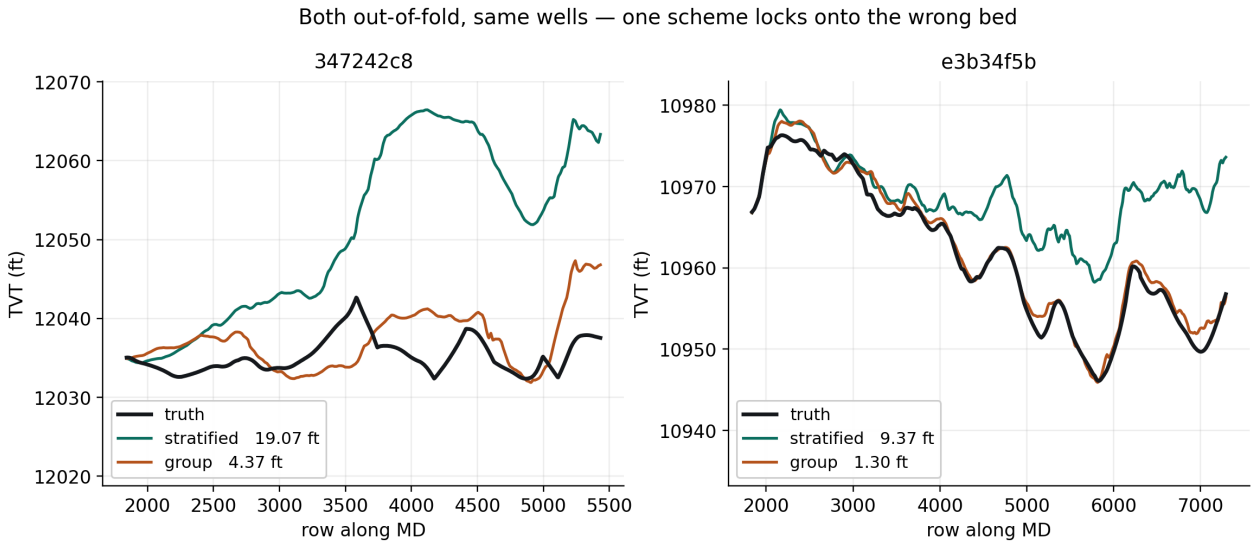


Metric	Value
Residual correlation	0.826
Blend at `w_group = 0.30`	**6.309** (flat over 0.20–0.40)
Per-well oracle selection	5.628
Wells where group wins	351 / 773
Overlap of each scheme's worst 30 wells	19 / 30

Mixing 30% of a group scheme that is 0.41 worse on average beats stratified alone.

On `347242c8`, stratified jumps to a different bed mid-well and scores 19.07 while group gets 4.37. On `e3b34f5b` it is 9.37 vs 1.30. Both are the same well and both are out-of-fold, so this is not a difference in fold difficulty — changing the split changes which evidence the model has to reason from.

A more extreme version of the same property is exact HMM forward-backward smoothing, which requires no training at all. It scores 11.94 on its own (the CNN is at 6.6) but has residual correlation of only 0.357, and contribute



d −0.22 to −0.28 on the LB.

Ablation: what did not work

All measured on 773-well OOF.

Method	Result	Mechanism
Post-hoc per-well gates in general	CV −0.148 → **LB +0.30**	The firing signal is a different quantity in OOF vs deployment
Quantile-rank firing suppression rule	LOCO −0.030 → **LB +0.197**	A threshold set on 773 wells fires on different wells among ~200
Seed ensemble of the same pipeline	8.410 (worse than the good seed alone, 8.365)	Residual correlation 0.992
Within-run checkpoint averaging	8.514 → 8.53	Same as above
Raw blending of a pseudo-typewell	+0.047	Heel and typewell GR differ by gain 0.80 / offset +16.8
Individually fixing HMM parameters	Every variant worse	Four errors are co-calibrated as a shrinkage estimator
Removing the 4 dip/azimuth channels	11.365 at ep90 (control 6.426)	All 9 channels contribute in this configuration

Method	Result	Mechanism
Post-hoc smoothing (savgol / median)	Best case −0.014	The decode is already smooth enough
Row-shift TTA (±4 / ±8)	+0.05 to +0.08	Shifting the input lattice phase does not resolve the aliasing

Appreciations

@sggpls for leading the team and for an enormous amount of work in the final week — the MeanShift spatial clustering that every validation scheme above is built on, and the tail-25 residual correction and ms7000 spatial folds that turned my raw 5-fold bank into `residual_spatial5`.

@bluepill for the PF ranker, the other half of our final submission. His solution was remarkably robust, and trusting CV over the public LB right to the very end is what got us the gold.

This leg in particular stands on two pieces of public work, without which there would have been nothing to iterate on:

- @tuckerarrants, whose public kernel is where the entire formulation comes from — well-as-image, the TVT lattice, column-wise CE, decoding by posterior expectation. Most of my competition was spent working out why my copy of his setup scored 1.5–2 ft worse than the original. His `smp-offline` dataset is also what supplies the ResNet34 ImageNet weights used here.
- @hengck23, whose MTP write-up is the source of the 9-channel input set this model consumes, and whose stream of public posts was the best running commentary in the competition.